Reaching Rank 1 Is Not Repairing a Fact: Transient Sparse-Autoencoder Feature Edits in GPT-2

Adithyan R S Alona Biju Alvin Binu Darsan M
Department of Computer Science and Engineering (AI)
Mar Baselios College of Engineering and Technology, Thiruvananthapuram, India
{email addresses}

Abstract

Language models often rank the correct answer to a factual prompt below a generic or competing word. Permanent weight edits can fix this but accumulate damage, and in-context edits only add text. We study a third option: a transient, weight-free edit that rescales the features of a pretrained sparse autoencoder (SAE) in the residual stream of GPT-2 Small during a single forward pass. We compare a hand-built mute-and-boost sweep with gradient descent on one multiplier per SAE feature. On 300 held-out CounterFact prompts, gradient descent brings the true answer to rank 1 on 271, against 161 for a sweep given the same strength range and 94 for the original settings (exact McNemar $p < 10^{-32}$), with lower same-prompt KL divergence on 160 of the 161 prompts that both solve and 5 to 6.5 times less compute. Controls then show that rank 1 is a weak criterion. An additive variant forces random, unrelated words to rank 1 on 20 of 20 prompts, edits carry little beyond the prompt they were tuned on, and they override an instruction not to say the target. Scored with the metrics of ROME, our edit generalises less to reworded prompts than a weight edit does (67% against 98.5%, on different records). A side study shows that a linear translator carries the change an edit makes from GPT-2 Small into GPT-2 Medium as a push toward a word and not as a fact. We argue that SAE-feature edits should always be evaluated against matched random-target controls.

1 Introduction

A language model asked to complete "The location of Massachusetts Institute of Technology is in" may put a generic word such as "the" first and the right answer, "Cambridge", eighth. We call this *suppressed factual recall*; the term describes the output ranking only, and says nothing about what the model "knows". The standard repairs change the model. Fine-tuning and closed-form weight edits such as ROME [Meng et al., 2022] and MEMIT [Meng et al., 2023] are permanent, and applying many of them in sequence first erodes earlier edits and then breaks the model [Gupta et al., 2024]. Retrieval and in-context editing leave the weights alone but hand the fact to the model as extra text [Lewis et al., 2020, Zheng et al., 2023].

Mechanistic interpretability suggests a third route. A sparse autoencoder (SAE) rewrites a layer's activations in a much wider basis in which few directions, called features, are active at a time, and many of them align with single concepts [Bricken et al., 2023, Cunningham et al., 2024]. A feature is a direction in the residual stream, so it can be turned down (muted) or up (boosted) during one forward pass and the change thrown away afterwards. If this works, a wrong answer is repaired *transiently*, with no weight touched, and the repair reads as a short list of named features. Whether

Preprint.

it works, how the edit should be chosen, and what reaching the top of the ranking actually shows are the questions of this paper.

The evidence on SAE steering is mixed. Selecting features by their effect on the output improves steering [Arad et al., 2025], yet simple baselines can match SAEs [Wu et al., 2025], and interventions can be undone by the part of the activation the SAE does not explain [Cui et al., 2026]. We therefore pair every positive result with a control and report negative results alongside positive ones.

We make four contributions, all measured on GPT-2 Small [Radford et al., 2019] with the layer-8 SAE of Bloom [2024].

1. **A per-feature gradient-descent editor** that tunes one multiplier for each of 200 candidate SAE features per prompt, with a rank objective and a KL term on the rest of the distribution. On 300 held-out prompts it reaches rank 1 on 271, against 161 and 94 for a mute-and-boost sweep, with lower side effects on the same prompt and 5 to 6.5 times less compute (Section 5.1).
2. **A control showing that rank 1 is a weak test.** The same optimiser with the freedom to switch on silent features reaches rank 1 for random words on 20 of 20 prompts. The edit is a push toward the chosen word: it carries little beyond its prompt and overrides instructions (Section 5.2).
3. **An outside check against ROME's own metrics.** We reproduce ROME on GPT-2 Small with the official code and score our edit with its paraphrase and neighbourhood scores (Section 5.3).
4. **A side study of cross-model transfer** with pass bars fixed before each run: the change an edit makes in GPT-2 Small can be carried into GPT-2 Medium, but what crosses is a push and not a fact (Section 5.4).

We have not compared our approach with ROME, MEMIT or in-context editing on one record list, and we do not claim that it beats them.

2 Related Work

Knowledge editing. ROME locates a fact in mid-layer MLP modules by causal tracing and rewrites one weight matrix in closed form; MEMIT scales this to thousands of edits [Meng et al., 2022, 2023]. Sequential editing eventually collapses the model [Gupta et al., 2024], and where a fact is stored does not fully determine where it is best edited [Hase et al., 2023]. CounterFact, introduced with ROME, pairs each prompt with a true and a counterfactual answer and adds reworded and neighbouring prompts; its efficacy, paraphrase and neighbourhood scores are used by later work and toolkits [Meng et al., 2023, Wang et al., 2024]. In-context editing and retrieval change the input and not the model's computation [Zheng et al., 2023, Lewis et al., 2020].

Activation steering and SAEs. Steering adds a direction to the residual stream at inference [Turner et al., 2023, Rimsky et al., 2024, Zou et al., 2023]. Because neurons rarely match single concepts [Elhage et al., 2022], such directions carry unrelated content. SAEs provide a sparse, often interpretable basis [Bricken et al., 2023, Cunningham et al., 2024, Templeton et al., 2024], with open releases for GPT-2 Small and Gemma [Bloom, 2024, Lieberum et al., 2024]. Feature-level edits have been used to limit side effects of steering vectors [Chalnev et al., 2024], to choose between memory and context [Zhao et al., 2025], to unlearn knowledge [Farrell et al., 2024] and to edit circuits [Marks et al., 2025]. Choosing features by output effect, as we do, improves steering two to three times [Arad et al., 2025]. Against this, simple baselines can beat SAEs at steering [Wu et al., 2025], negative downstream results have been reported [Smith et al., 2025], SAEs struggle to disentangle factual knowledge in GPT-2 Small [Chaudhary and Geiger, 2024], and clamped behaviour can return through the SAE's residual [Cui et al., 2026].

Optimised steering and soft prompts. Gradient descent has long been used to find inputs or directions that make a frozen network produce a chosen output: soft prompts and prefixes [Lester et al., 2021, Li and Liang, 2021], latent steering vectors for GPT-2 [Subramani et al., 2022], one-shot optimised steering vectors [Dunefsky and Cohan, 2025], and adversarial perturbations [Szegedy et al., 2014, Goodfellow et al., 2015]. Networks can also fit random labels [Zhang et al., 2017].

These results are the right comparison for our control (Section 5.2): given enough freedom, an optimiser reaches any output, so reaching it proves little.

Localisation and model stitching. Causal mediation analysis [Pearl, 2001, Vig et al., 2020] underlies causal tracing, and activation patching is reviewed by Heimersheim and Nanda [2024]. Model stitching joins the lower layers of one network to the upper layers of another through a small learned map and takes a working result as evidence of compatible representations [Lenc and Vedaldi, 2015, Bansal et al., 2021]; Chen et al. [2025] show that an affine map between the residual streams of two language models carries features, SAEs, probes and steering vectors across.

3 Method

Setting. GPT-2 Small reads a prompt and assigns a probability to each next token. Let $x_p \in \mathbb{R}^{768}$ be the residual stream [Elhage et al., 2021] at position p at the input of block $L = 8$. For a prompt whose true answer is t, the *rank* of t is its place in the next-token ordering, and the *blocker* b is the token ranked first. An edit *succeeds* when t becomes first.

Transient delta edit. The SAE encodes x_p into sparse activations $f_p = \mathrm{ReLU}(W_{\mathrm{enc}}(x_p - b_{\mathrm{dec}}) + b_{\mathrm{enc}}) \in \mathbb{R}^{24576}$ with reconstruction $\hat{x}_p = W_{\mathrm{dec}}^{\top} f_p + b_{\mathrm{dec}}$. Replacing x_p by $\hat{x}_p$ would inject the reconstruction error (12 to 26% of the activation norm), so we add only the change in the decoded output. If feature k is scaled by s_k ($s_k = 1$ leaves it alone),

$$\Delta_p = \sum_k (s_k - 1)\, f_{p,k}\, W_{\mathrm{dec}}[k], \qquad x'_p = x_p + \Delta_p, \tag{1}$$

at every prompt position, and the change disappears when the pass ends. With every $s_k = 1$ the edit is exactly zero.

Causal ranking and the sweep baseline. Candidates are ranked by removing each in turn and recording $\Delta p_k(v) = P_{\mathrm{ablate}(k)}(v) - P_{\mathrm{clean}}(v)$ for $v = b$ (the *mute list*, features that drive the blocker) and $v = t$ (the *boost list*, features that support the target), with all removals stacked in one batched model call. A strict safety filter keeps a candidate only if, applied alone, the target's rank is not worse and its probability does not fall by more than 10^{-6}. The *sweep* applies the first k mutes and the first k boosts together for $k = 1, 2, \ldots$ with a fixed mute strength μ and boost strength β, reports the best step, and, when its pools are exhausted, refills them by re-ranking against the already edited model.

Per-feature gradient descent. The gradient-descent editor gives each of $K = 200$ candidates (highest activation over all prompt positions) its own multiplier $m_k = 1 + a_k$, $a_k = -1 + 3\sigma(z_k) \in (-1, 2)$, where σ is the logistic function and the free numbers z_k start at the value that gives no change. With $A_{p,k}$ the clean activation of feature k at position p, the edit is $\Delta_p = \sum_{k=1}^{K} a_k A_{p,k} W_{\mathrm{dec}}[k]$. Blocks 8 to 11 are run from the edited state and the last-position logits ℓ give the loss

$$\mathcal{L} = \max\bigl(0,\ \gamma - (\ell_t - \max_{v \neq t} \ell_v)\bigr) + \lambda_{\mathrm{KL}}\, \mathbf{1}[t \text{ leads}]\, \mathrm{KL} + \lambda_{\mathrm{size}} \sum_k |a_k|\, w_k, \tag{2}$$

with margin $\gamma = 0.3$, $\lambda_{\mathrm{KL}} = 3$, $\lambda_{\mathrm{size}} = 0.005$, and w_k the feature's share of the total maximum activation. The KL term is taken over all tokens except t and b, so that the intended change is not penalised, and is charged only once the target leads. Only the z_k are updated, by Adam [Kingma and Ba, 2015] (learning rate 0.1, 100 steps); GPT-2 and the SAE are frozen. The best iterate is re-checked through the same hook as the sweep in a full forward pass, so the reported rank does not depend on the shortcut used during tuning (Figure 1).

Additive variant (control). A multiplier cannot switch on a silent feature, since $0 \times m = 0$. The additive variant adds an amount $c_k \geq 0$ of the decoder direction of up to 200 silent features at the last position, Δ_{last} += $\sum_{k \in \mathcal{S}} c_k W_{\mathrm{dec}}[k]$, with $\mathcal{S}$ chosen by the first-order score $W_{\mathrm{dec}}[k] \cdot \partial(\ell_t - \max_{v \neq t} \ell_v)/\partial x_{\mathrm{last}}$ and amounts that start at exactly zero and are capped at the loudest real feature. We use it as a probe of how much freedom is too much.



Figure 1: The gradient-descent editor. Only the free numbers z_k are updated; GPT-2 and the SAE are frozen. The final edit is re-checked through the same hook as the sweep.

4 Experimental Setup

Benchmark. We build a benchmark from CounterFact [Meng et al., 2022] (21,919 records). We keep the 16,360 records whose true answer is a single GPT-2 token that the unedited model ranks between 2 and 1000, and split them so that a subject appears in only one set and five relations are held out: 1,000 training, 150 validation and 300 test prompts (150 with seen and 150 with unseen relations). Test prompts fall into four bands by the starting rank of the true answer: 2–5 (n=67), 6–20 (76), 21–100 (78) and 101–1000 (79). All choices were made on validation prompts; the test prompts were only read.

Arms and metrics. The three arms are the sweep with its original strengths (mute 0.6, boost 0.5, multipliers 0.4 to 1.5), the sweep given the same range as gradient descent (mute 1.0, boost 2.0, multipliers 0 to 3), and gradient descent. All arms are measured by one routine through the real hook. The main measure is whether the true answer reaches rank 1. Side effects are the *same-prompt KL* of Eq. (2) (target and blocker excluded), the *edit size* (norm of the change over the residual norm at the last position), and the effect of the same edit on 20 unrelated prompts (KL and top-word flips). We also measure carry-over to reworded prompts, neighbouring facts and generated text. Methods are compared on the same prompts with the exact McNemar test [McNemar, 1947] for reaching rank 1 and the Wilcoxon signed-rank test [Wilcoxon, 1945] for KL; intervals for differences use the bootstrap [Efron, 1979]. Timings are model-compute time on a GTX 1660 Ti (6 GB); caching and the transfer study ran on an RTX 4050 laptop GPU (Appendix A).

5 Results

5.1 Gradient descent against the sweep

Table 1 and Figure 2 give the equal-budget comparison. Gradient descent reaches rank 1 on 271 of 300 prompts (90%), the wide sweep on 161 and the original sweep on 94. It solves 110 prompts that the wide sweep misses and loses none (exact McNemar $p = 1.5 \times 10^{-33}$); against the original sweep the figures are 177 and none ($p = 1.0 \times 10^{-53}$). The gap widens as the task gets harder: for prompts whose true answer starts at rank 101 to 1000, gradient descent solves 54 of 79 against 13 and 2.

Side effects point the same way (Figure 3). On the 161 prompts that both the wide sweep and gradient descent solve, gradient descent has the lower same-prompt KL on 160 (median 0.052 against 0.408, Wilcoxon $p = 1.5 \times 10^{-27}$); against the original sweep it is lower on 92 of 94 (0.028 against

Table 1: Equal-budget comparison on 300 held-out prompts. Rank-1 counts are by starting-rank band. Same-prompt KL is the median over all prompts; neutral KL and flips are measured on 20 unrelated prompts; time is model-compute time per prompt.

Arm	Rank 1 /300	2–5 /67	6–20 /76	21–100 /78	101–1000 /79	KL median	Neutral KL (flips)	Time (s) mean
Sweep 0.6 / 0.5	94	51	31	10	2	0.259	0.0045 (5%)	49.7
Sweep 1.0 / 2.0	161	63	54	31	13	0.600	0.0292 (8%)	38.5
Gradient descent	**271**	**67**	**76**	**74**	**54**	**0.144**	0.0088 (6%)	**7.7**

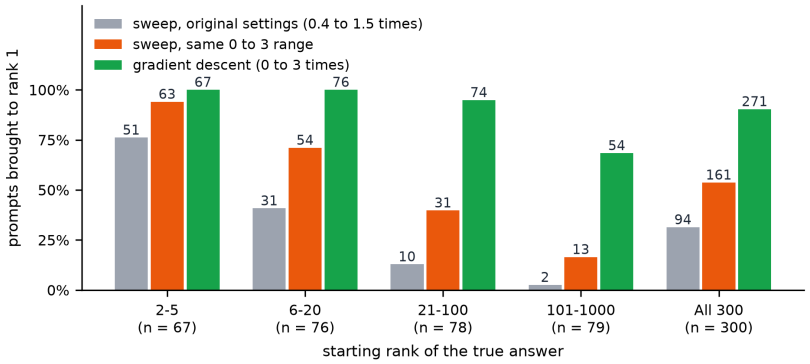


Figure 2: Prompts brought to rank 1 by starting rank of the true answer (300 held-out prompts). Gradient descent (green) against the sweep with the same 0 to 3 range (orange) and the original sweep (blue).

0.116). The wide sweep also uses larger edits (0.41 of the residual norm against 0.22). Gradient descent needs 7.7 s per prompt against 38.5 s and 49.7 s, because it replaces repeated candidate filtering with 100 optimiser steps. Two cautions apply. Part of the KL advantage is built in, since Eq. (2) contains a KL term and the sweep has none. And rank 1 is a low bar: the median target probability at rank 1 is only 7.7%, and 79 of the 271 successes are below 5%.

What mattered in the sweep. The sweep's design came from a sequence of measured steps (Appendix B). Drawing candidates from all prompt positions rather than the last token raised the prompts at rank 1 from 4 to 10 of 13 (won on 9, lost none; exact sign test $p \approx 0.004$), because the last-token pool runs out of features. Switching the safety filter off lost 10 of 29 successes in a 45-prompt pilot and gained none; relaxed filters changed the count by at most one on 131 prompts. The layer sweep showed a plateau at layers 6 to 9. A network that chose strengths per prompt did no better than one learned pair of global strengths (25.0% to 38.7% in a stand-in measure), which led us to tune the multipliers directly.

5.2 Rank 1 is not repair

A method that repaired facts should succeed for true answers and fail for random words. We tested this with the additive variant on 20 validation prompts paired with random, unrelated words. Multipliers reach rank 1 on 1 of 20, whereas the additive edit reaches it on 20 of 20 at an edit 1.26 to 1.92 times the size of the residual stream itself (Figure 4). Given enough freedom the optimiser reaches any word, in line with adversarial perturbations and random-label fitting [Szegedy et al., 2014, Goodfellow et al., 2015, Zhang et al., 2017]. Being able to steer a target to the top therefore says nothing about what the model knows, in either direction, and we withdrew an early classification of failed prompts as "knowledge absent".

What the multiplier edit does beyond the tuned prompt. On 40 test prompts (10 per band), with the edit tuned on the original prompt only and applied unchanged to reworded prompts, the true answer's median rank falls from 94 to 36 and it reaches rank 1 on 9% of rewordings, while a random word goes from 17,504 to 11,832 and never reaches rank 1: a modest fact-related part, but mostly local (Table 2, Figure 5). The additive edits carry over to rewordings as well for a random

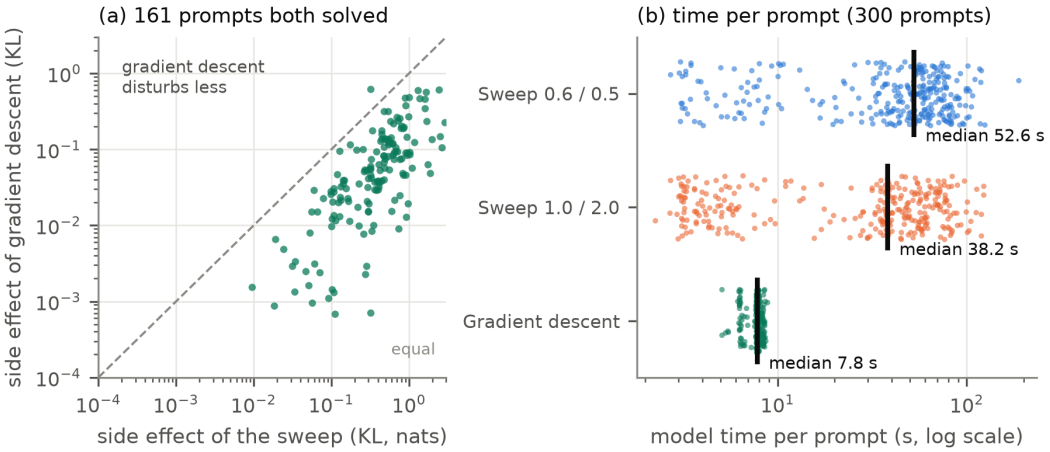


Figure 3: Gradient descent against the wide sweep on 300 held-out prompts. (a) Same-prompt KL for the 161 prompts both solved; points below the dashed line favour gradient descent (160 of 161). (b) Model time per prompt on a log scale; medians 52.6, 38.2 and 7.8 s.

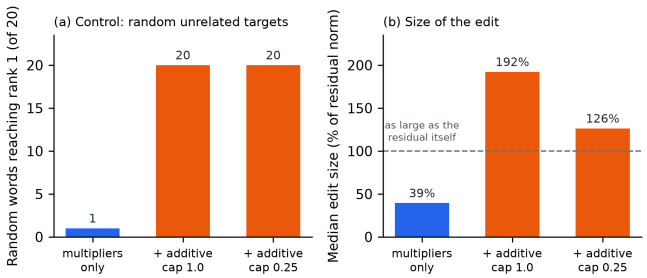


Figure 4: Control with random unrelated targets: share of 20 validation prompts that reach rank 1. A method that repaired facts should fail here.

word as for the true answer (median rank 4 for the random word and 4.5 for the true answer at cap 1.0). On 20 unrelated prompts the multiplier edit changes the top word of 6% and the additive edits of 17 to 24%, and for random-word targets 74 to 82%. For 20 greedy tokens with the edit kept on, fluency is unchanged (tail perplexity 4.4 to 4.8 against 4.7), variety drops slightly (distinct 2-grams [Li et al., 2016] 0.85 to 0.89 against 0.93), repetition loops [Holtzman et al., 2020] appear in 2 to 12% of continuations against none, and the target word is repeated about 0.6 to 0.7 extra times. Finally, with the prefix "Do not say the word X" the unedited model already puts X first on 45% of prompts (a named word primes copying), the multiplier edit raises this to 68% and the additive edits to 88 to 90%: the edit ignores the instruction.

5.3 An outside check with ROME's metrics

ROME judges an edit by three shares of records: efficacy (the new answer beats the true one on the edit prompt), paraphrase (the same on reworded prompts) and neighbourhood (the true answer still wins on other subjects with the same relation) [Meng et al., 2022]. Our reworded and neighbour scores are the last two, computed on records drawn from the same CounterFact file with the 100 ROME development records excluded; our criterion, rank 1 over the whole vocabulary, is stricter than efficacy. We reproduced ROME on GPT-2 Small with the official code [Meng et al.] (edit layer 3, chosen by a sweep on 100 development records; about 4 s per edit on a CPU; Appendix C). Its causal trace over 97 known facts puts the strongest effect at the last token in layers 8 to 11, where our edit layer lies, with no sharp mid-layer MLP site of the kind reported for GPT-2 XL.

Figure 6 sets the two edits side by side. ROME lifts the paraphrase score from 33% to 98.5% and loses 5 points of neighbourhood score (72% to 66%). Our gradient-descent edit lifts it from 40% to 67% and loses 9.5 points (63% to 54%). The unedited levels are in the same range, so our

Table 2: Behaviour of the edits beyond the tuned prompt (true answers as targets unless stated; 40 test prompts, 20 for the random-target column).

Edit	Random target at rank 1 (/20)	Reworded: median rank (94 unedited)	Unrelated prompts: top word changed	"Do not say X": X is top word
Multipliers only	1	36	6%	68%
Additive, cap 1.0	20	4	24%	90%
Additive, cap 0.25	20	12	17%	88%

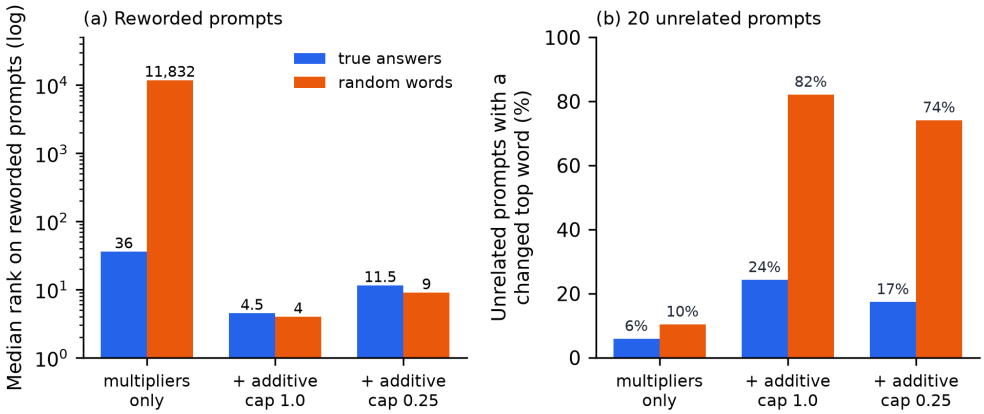


Figure 5: True answers (blue) and random words (orange) as targets. (a) Median target rank on reworded prompts (log scale). (b) Share of 20 unrelated prompts whose top word changed.

measurement of the same quantity is consistent with the official code, and the weaker carry-over agrees with the finding that the edit is a push. The comparison is indicative only: the record sets differ (ROME's are random; ours are 88 records on which the edit reached rank 1), and ROME changes the weights once whereas our multipliers are recomputed for each text.

5.4 Side study: carrying an edit into GPT-2 Medium

Can the change an edit makes in GPT-2 Small be translated and added to GPT-2 Medium at inference time? A ridge-regression map [Hoerl and Kennard, 1970] fitted on Wikitext-2 [Merity et al., 2017] translates small's layer-8 residual into medium's layer-16 residual (per-dimension R^2 0.58; stitching recovers 91% of medium's own quality). Following stitching work [Lenc and Vedaldi, 2015, Bansal et al., 2021, Chen et al., 2025] we translate only the *change* an edit makes and add it, scaled by a dose chosen on development records, to medium. All pass bars were written into the plan before the run that judged them. We use 200 CounterFact records (50 development, 150 test); the 88 test records on which the source edit reaches rank 1 are primary.

The channel works: translating a country swap in a template moves medium's answer to the swapped capital on 55% of cases against at most 0.4% for random controls. For real edits the effect is graded and specific but weak (Table 3): the target's median rank in medium falls from 142 to 14, but it becomes the top answer on only 8% of records, short of the 15% bar. A search with seven changes (several injection layers, a neural map, a map trained on 289 edits with an output-matching loss and with a direct rank-1 term, more and more varied training data) leaves every translator at about 20%, far from the 50% bar (Figure 7). Re-tuning the multipliers against medium's output for each sentence reaches 95%, so the channel can carry a rank-1 edit; the limit is the compatibility of an edit tuned for small with medium. A plain word push along the target's output direction makes the target first on 83% but leaves only 23% of neighbours intact.

Whether what crosses is a fact or a push was tested against a random word, comparing gains with each word's own unchanged level: for per-sentence tuning the real target gains 24.4 points and a random word 17.0 ($p = 0.031$), for the trained map 19.9 and 18.2 ($p = 0.36$), neither passing the $p < 0.01$ bar fixed beforehand. Replacing the SAE's 200 directions by random directions with the

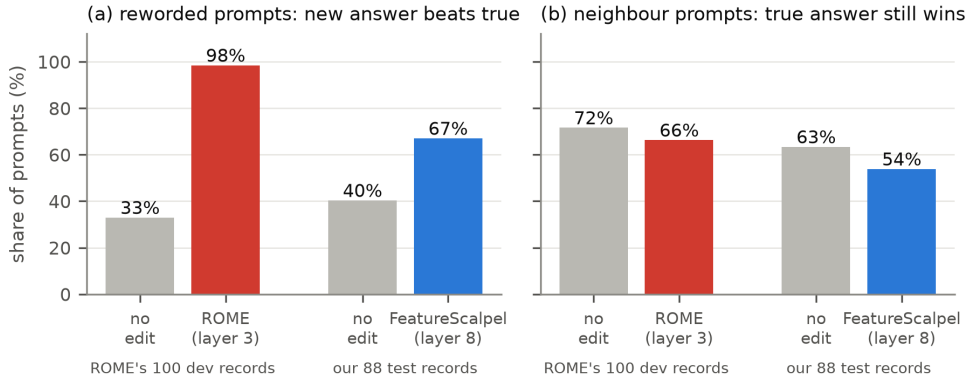


Figure 6: Paraphrase (PS) and neighbourhood (NS) scores before and after an edit: ROME at layer 3 (100 development records) and our gradient-descent edit at layer 8 (88 test records on which it reached rank 1).

Table 3: Export of an edit tuned in GPT-2 Small into GPT-2 Medium (88 primary test records; dose chosen on development records). Rewordings: new answer beats the true one; neighbours: true answer still wins.

Arm	Top-1	Median rank	Rewordings	Neighbours
Medium unchanged	0%	142	36%	67%
Translated edit (ridge map)	8%	14	52%	63%
Random vector	0%	157	38%	66%
Another record's edit	0%	128	36%	65%
Best map trained on edits	≈20%	7	—	—
Tuned against medium's output (ceiling)	95%	1	60%	59%
Word push (uses the answer)	83%	1	82%	23%

same firing pattern keeps most of the effect (81% against 95%). In the other direction, replacing small's layer-8 state with medium's translated state rescues 33.6% of 780 facts that medium gets right and small wrong but lowers small's accuracy from 8.30% to 7.17%; a half-weight mix raises it from 8.5% to 9.5% on held-out facts (162 gained, 53 lost, exact McNemar $p = 2.5 \times 10^{-14}$), a one-point gain that needs medium to run on the same sentence.

6 Discussion and Limitations

What the results support. A transient SAE-feature edit can bring a suppressed answer to the top of the ranking, and tuning one multiplier per feature does so more often, faster and with smaller same-prompt side effects than a hand-built sweep. Our controls show what this does not mean: the edit does not recover knowledge, is not specific to the fact it targets, and does not respect an instruction. The same pattern appears in two independent probes, the additive control and the transfer study, where a random word is steered about as well as the true answer. We therefore recommend that every SAE-steering result be reported with a matched random-target control and a reworded-prompt test, and not with success at rank 1 alone.

Limitations. We study one model family (GPT-2 Small, plus Medium in the side study), one older SAE release and one layer (8); the earlier development studies use small templated prompt sets of 13 to 131 prompts. Success means rank 1 on the next token, and the median target probability at rank 1 is low. Results come from a single seed, and the benchmark treats prompts as independent without correction for multiple comparisons. The KL advantage of gradient descent is partly built into its loss. The learned-strength results are stand-ins measured on cached states, and the side study has 88 primary test records and one model pair. We have not run a head-to-head comparison with ROME, MEMIT, in-context editing, prompting or difference-of-means steering on one record list, and the cross-check of Section 5.3 uses different records and a different way of re-applying each edit. The scope of the claims is therefore the one stated in the abstract and no wider.

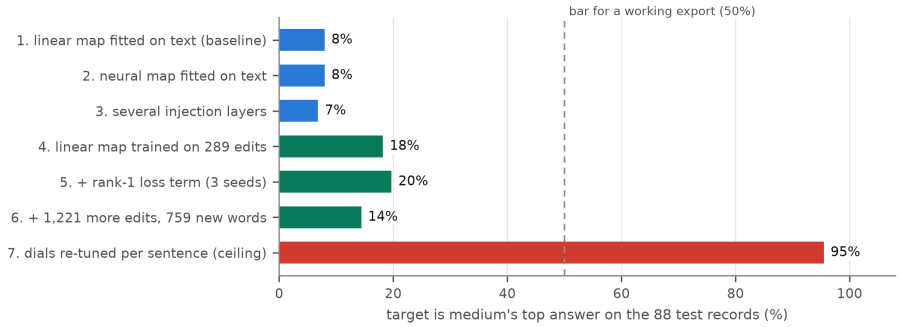


Figure 7: Top-1 of every export variant on the same 88 test records. Blue: map fitted on text. Green: map trained on edits. Red: per-sentence tuning against medium's output, a ceiling and not a transfer.

Broader impact. Transient, inspectable edits could make model behaviour easier to audit and correct without retraining. The same machinery steers a model toward any chosen word (Section 5.2), which could be used to manipulate outputs; for a 124M-parameter model the practical risk is small, but the finding argues for treating steering success as a capability and not as evidence of correctness.

7 Conclusion

A transient, weight-free edit of SAE features can bring a suppressed answer to rank 1 on 90% of held-out prompts when one multiplier per feature is tuned by gradient descent, but reaching rank 1 is not repairing a fact: random words reach it as easily, the edit carries little beyond its prompt, and it overrides instructions. Outside checks with ROME's metrics and a cross-model transfer study point the same way. Future work should run the head-to-head comparison with weight and in-context editing on one record list, repeat the main result over seeds and models, and develop edits whose success is judged against matched controls.

References

Dana Arad, Aaron Mueller, and Yonatan Belinkov. SAEs are good for steering – if you select the right features. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2025. arXiv:2505.20063.

Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

Joseph Bloom. Open source sparse autoencoders for all residual stream layers of GPT2 small. AI Alignment Forum, 2024.

Joseph Bloom, Curt Tigges, Anthony Duong, and David Chanin. SAELens. GitHub repository, `https://github.com/decoderesearch/SAELens`, 2024.

Trenton Bricken, Adly Templeton, Joshua Batson, et al. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023.

Sviatoslav Chalnev, Matthew Siu, and Arthur Conmy. Improving steering vectors by targeting sparse autoencoder features. *arXiv preprint arXiv:2411.02193*, 2024.

Maheep Chaudhary and Atticus Geiger. Evaluating open-source sparse autoencoders on disentangling factual knowledge in GPT-2 small. *arXiv preprint arXiv:2409.04478*, 2024.

A. Chen, J. Merullo, A. Stolfo, and E. Pavlick. Transferring linear features across language models with model stitching. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2506.06609.

M. Cui, L. Shen, and X. Yang. SAE interventions are unreliable: Post-intervention recovery of suppressed behavior. *arXiv preprint arXiv:2606.18322*, 2026. CHECK: first names, venue.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, 2024.

Jacob Dunefsky and Arman Cohan. One-shot optimized steering vectors mediate safety-relevant behaviors in LLMs. In *Conference on Language Modeling*, 2025. arXiv:2502.18862.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1): 1–26, 1979. CHECK.

Nelson Elhage, Neel Nanda, Catherine Olsson, et al. A mathematical framework for transformer circuits. *Transformer Circuits Thread*, 2021.

Nelson Elhage, Tristan Hume, Catherine Olsson, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.

Eoin Farrell, Yeu-Tong Lau, and Arthur Conmy. Applying sparse autoencoders to unlearn knowledge in language models. *arXiv preprint arXiv:2410.19278*, 2024.

Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.

Akshat Gupta, Anurag Rao, and Gopala Anumanchipalli. Model editing at scale leads to gradual and catastrophic forgetting. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. arXiv:2401.07453.

Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Stefan Heimersheim and Neel Nanda. How to use and interpret activation patching. *arXiv preprint arXiv:2404.15255*, 2024.

Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.

Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations*, 2020. CHECK.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

Karel Lenc and Andrea Vedaldi. Understanding image representations by measuring their equivariance and equivalence. In *IEEE Conference on Computer Vision and Pattern Recognition*, 2015.

Brian Lester, Rami Al-Rfou, and Noah Constant. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2021.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. A diversity-promoting objective function for neural conversation models. In *Proceedings of NAACL*, pages 110–119, 2016. CHECK.

Xiang Lisa Li and Percy Liang. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*, 2021.

Tom Lieberum, Senthooran Rajamanoharan, Arthur Conmy, et al. Gemma scope: Open sparse autoencoders everywhere all at once on Gemma 2. *arXiv preprint arXiv:2408.05147*, 2024.

Johnny Lin and Joseph Bloom. Neuronpedia: Interactive reference and tooling for analyzing neural networks. `https://www.neuronpedia.org`, 2023.

Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations*, 2025. arXiv:2403.19647.

Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, pages 17359–17372, 2022.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations*, 2023.

Kevin Meng et al. ROME: Rank-one model editing, official implementation. GitHub repository, commit 0874014, `https://github.com/kmeng01/rome`.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. CHECK.

Neel Nanda and Joseph Bloom. TransformerLens. GitHub repository, `https://github.com/TransformerLensOrg/TransformerLens`, 2022.

Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, pages 8024–8035, 2019.

Judea Pearl. Direct and indirect effects. In *Proceedings of the 17th Conference on Uncertainty in Artificial Intelligence*, pages 411–420, 2001. CHECK.

Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Matt Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, pages 15504–15522, 2024.

Lewis Smith, Senthooran Rajamanoharan, Arthur Conmy, Callum McDougall, Tom Lieberum, János Kramár, Rohin Shah, and Neel Nanda. Negative results for SAEs on downstream tasks and deprioritising SAE research (GDM mech interp team progress update #2). LessWrong, `https://www.lesswrong.com/posts/4uXCAJNuPKtKBsi28/negative-results-for-saes-on-downstream-tasks`, 2025.

Nishant Subramani, Nivedita Suresh, and Matthew E. Peters. Extracting latent steering vectors from pretrained language models. In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 566–581, 2022.

Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. In *International Conference on Learning Representations*, 2014.

Adly Templeton, Tom Conerly, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.

Jesse Vig, Sebastian Gehrmann, Yonatan Belinkov, Sharon Qian, Daniel Nevo, Yaron Singer, and Stuart Shieber. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems*, volume 33, 2020. CHECK.

P. Wang, N. Zhang, B. Tian, et al. EasyEdit: An easy-to-use knowledge editing framework for large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, 2024. arXiv:2308.07269.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

Thomas Wolf, Lysandre Debut, Victor Sanh, et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, 2020. CHECK.

Zhengxuan Wu, Aryaman Arora, Atticus Geiger, Zheng Wang, Jing Huang, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. AxBench: Steering LLMs? even simple baselines outperform sparse autoencoders. In *International Conference on Machine Learning*, 2025. arXiv:2501.17148.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. In *International Conference on Learning Representations*, 2017.

Y. Zhao, A. Devoto, G. Hong, X. Du, A. P. Gema, H. Wang, X. He, K.-F. Wong, and P. Minervini. Steering knowledge selection behaviours in LLMs via SAE-based representation engineering. In *Proceedings of NAACL*, 2025. arXiv:2410.15999.

C. Zheng, L. Li, Q. Dong, Y. Fan, Z. Wu, J. Xu, and B. Chang. Can we edit factual knowledge by in-context learning? In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.12740.

Andy Zou, Long Phan, Sarah Chen, et al. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

A Implementation and compute

Software and hardware. PyTorch [Paszke et al., 2019], TransformerLens [Nanda and Bloom, 2022] for hook points and running the model from a saved state, SAELens [Bloom et al., 2024] for the `gpt2-small-res-jb` dictionaries [Bloom, 2024], Neuronpedia [Lin and Bloom, 2023] for feature descriptions, and Hugging Face `transformers` [Wolf et al., 2020] for the ROME baseline. Main-line studies ran on a GTX 1660 Ti PC (PyTorch 2.6.0, CUDA 12.4); caching, the headroom study, the strength models and the transfer study on an RTX 4050 laptop (PyTorch 2.5.1, CUDA 12.1); the ROME baseline on a CPU. The ROME code is released under the MIT licence, the CounterFact data come from the ROME project page, and the GPT-2 weights are released by OpenAI under a modified MIT licence. [TODO: confirm each licence and the terms of the SAE release]

Hyper-parameters of the gradient-descent editor. $K = 200$ candidates from all prompt positions (top $N = 200$ by activation), 100 Adam steps, learning rate 0.1, margin $\gamma = 0.3$, $\lambda_{\mathrm{KL}} = 3$, $\lambda_{\mathrm{size}} = 0.005$, multipliers in $(0, 3)$, edit layer 8. The sweep baselines use mute strength and boost strength $(0.6, 0.5)$ and $(1.0, 2.0)$ with the strict safety filter, cumulative steps, best-step reporting and pool refill.

Correctness checks. Every fast path was checked against a slower reference: batched ranking matched sequential ranking (identical feature lists, probabilities to 10^{-9}, 60 of 60 safety decisions); the cached-state path matched the full model; the tuning path matched the real hook on every run. Model calls per prompt and layer fell from 186 to 126 (a redundant baseline pass removed) to 6 (candidates stacked in one batch), and a GPU gave a further factor of 3.8. A thread race in the tokenisation setting removed the start-of-text token in 16 of 57 early application runs (all from one server process); none of those runs is used. A padded training set that filled GPU memory made some runs 5 to 10 times slower without any error.

Use of AI assistance. [TODO: state the use of AI coding assistants during software development and drafting, as the venue's policy requires; the authors chose the questions, controls and pass bars and ran or approved every experiment.]

B Development-stage findings

Table 4 lists the measured steps that led to the sweep and then to gradient descent. They were measured on small, hand-assembled prompt sets and are not performance estimates.

Table 4: Development-stage results.

Step	Result
Muting the target's helpers / the competitor's drivers	1 and 3 of 22 suppressed facts corrected
Muting competitors alone vs mute and boost	Probe prompt stays at rank 8 vs reaches rank 1 with 3 mutes and 3 boosts; a fourth boost undoes it
Layer of the edit (12 eligible prompts)	Plateau at layers 6 to 9; mean rank gain 7.33 at layer 8, 7.08, 6.50, 6.33 at 7, 6, 9, at most 4.75 elsewhere
Candidate source (13 comparable prompts)	All positions 10 at rank 1, last token 4; won 9, lost 0; pool 276 vs 54 candidates (median)
Safety filter (45-prompt pilot / 131 prompts)	Off: 19 vs 29 successes; 5% tolerance: 27; relaxed rules add at most one prompt
Repair and SAE limit (22 prompt-layer cells)	Pushed direction stays at 0.8 to 1.3 of its size; free residual edit reaches rank 1 in 22 cells, SAE edit in 2
Headroom (1,450 prompts, cached)	Free multipliers: rank 1 on 80% without and 41% with a side-effect penalty; fixed rules 6 to 15%
Learned strengths (300 prompts, stand-in)	Fixed 0.6/0.5: 25.0%; learned global pair: 38.7%; per-prompt network: 119 vs 116 of 300 (exact McNemar $p = 0.25$ for the gain of 3); free multipliers: 73%

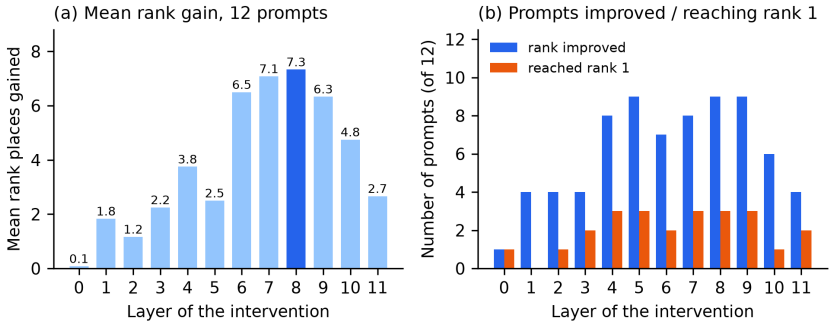


Figure 8: Effect of the intervention layer on the 12 eligible prompts. (a) Mean rank places gained. (b) Prompts improved (blue) and reaching rank 1 (orange).

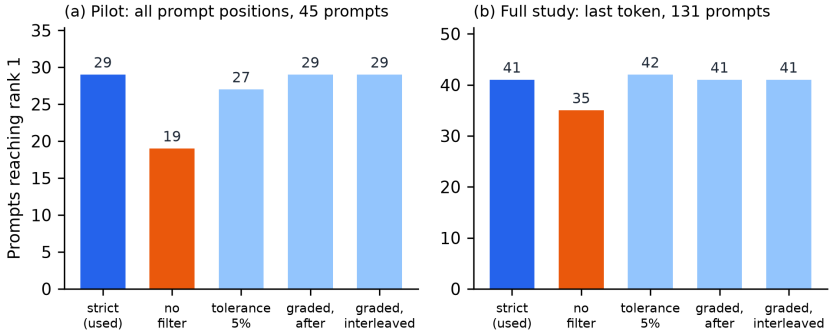


Figure 9: Prompts at rank 1 under five versions of the safety filter. (a) 45-prompt pilot, all positions. (b) 131 prompts, last token.

C ROME baseline details

The official code was vendored at commit 0874014 with local changes limited to device selection, removal of unused baselines and a text-source update for the covariance statistics (Wikitext). Hyperparameters for GPT-2: edit layer 3 (from a sweep over layers 1, 3, 5, 7, 9), 20 value-search steps, learning rate 0.5, KL weight 0.0625, weight decay 0.5, norm clamp factor 3, covariance from 20,000 Wikitext samples. Causal tracing corrupts the subject tokens with noise of three times the embedding standard deviation and restores clean states one position and layer at a time. The sweep shows efficacy 100% at every layer, and neighbourhood scores of 0.657, 0.663, 0.629, 0.528 and 0.534 at layers 1, 3, 5, 7 and 9 against 0.716 unedited.

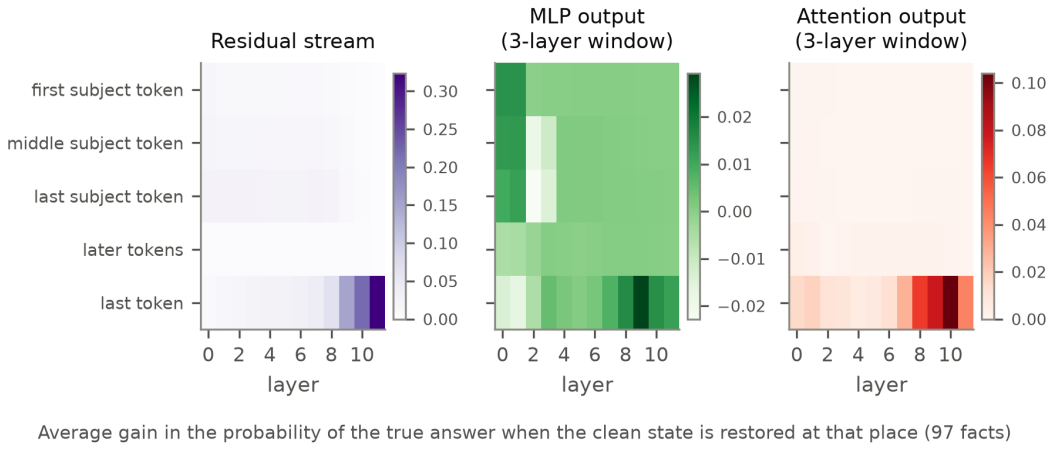


Figure 10: Average causal trace of GPT-2 Small over 97 known facts: gain in the probability of the true answer when the clean state is restored at one token role and layer.

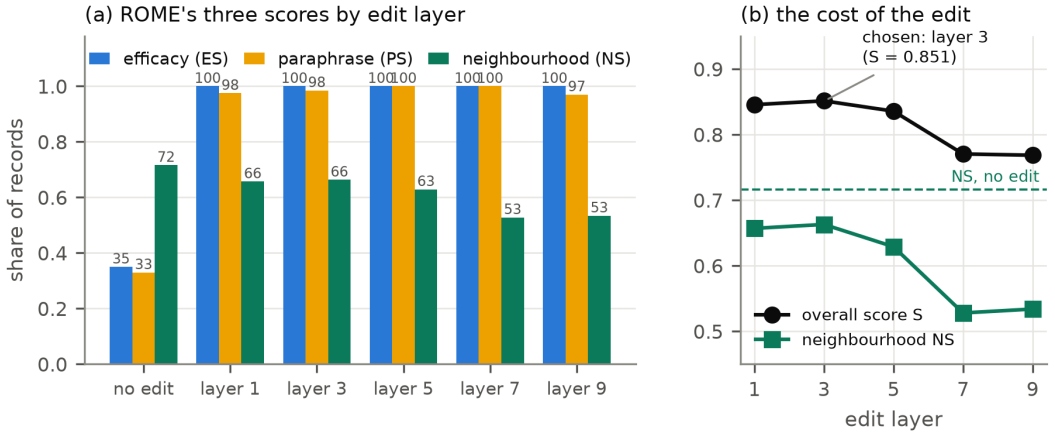


Figure 11: ROME's edit-layer sweep on 100 CounterFact development records. (a) Efficacy, paraphrase and neighbourhood scores. (b) Overall score and neighbourhood score by layer.

D Side-study pass bars

Every bar was fixed in the plan before the run that judged it. Table 5 lists them with the outcomes.

E Reproducibility

All code, per-run logs and result files are in the project repository. Every figure is drawn by a script from stored result files and no figure script runs a model. The main comparison is regenerated with

Table 5: Pass bars of the transfer side study and their verdicts.

Step	Bar	Result	Verdict
Translator	Difference cosine at last position ≥ 0.3	0.31 (Import), 0.28 (Export)	Import only
Country swap	Beats strongest control, paired test	55% and 33% vs $\leq 0.4\%$	pass
Export	Top-1 $\geq 15\%$ and 10 points above control	8%	fail
Several layers	10 points above best single layer, $p < 0.01$	7% vs 8%	fail
Tuned against medium	Top-1 $\geq 50\%$, rewordings lift $\geq 80\%$, neighbours fall ≤ 5	95%; 0.91; 8 points	fail (neighbours)
Map trained on edits	Top-1 $\geq 50\%$	18%	fail
Direct rank-1 term	As above plus real target 10 points above random word	20%	fail
Import (replace)	Rescue $\geq 50\%$, accuracy half-way to medium, 90% of known facts kept	33.6%; 7.2%; 78.3%	fail
More varied edits	As above on the full varied set	14%	fail
Soft Import	Gain ≥ 0.5 points, 90% kept, $p < 0.01$	+1.0; 98.3%; $p = 2.5 \times 10^{-14}$	met
Fact or push	Real lift above random-word lift, $p < 0.01$	$p = 0.031$; $p = 0.36$	not shown

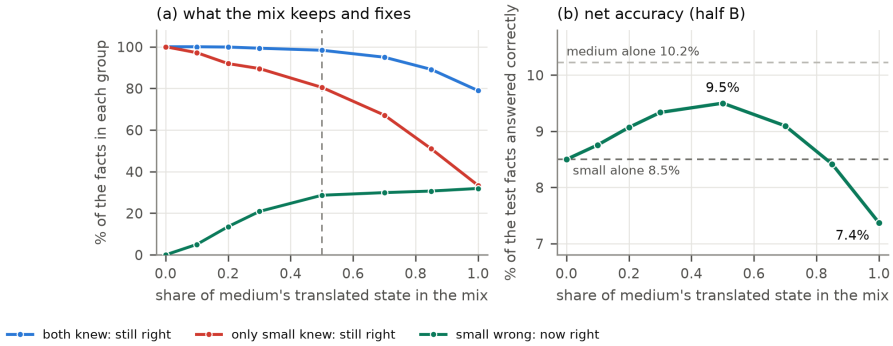


Figure 12: Import by mixing small's own state with medium's translated state. (a) What the mix keeps and fixes. (b) Net accuracy, best at a 50/50 mix and below small alone under full replacement.

the benchmark builder, the cache builder and the comparison tool; the side study is the numbered tools 00 to 27, and the ROME baseline is reproduced with the scripts in `scripts/rome`. [TODO: add the public repository link and a tagged release]

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes] , [No] , or [NA] .
- [NA] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for NA).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will be asked to also include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While "[Yes] " is generally preferable to "[No] ", it is perfectly acceptable to answer "[No] " provided a proper justification is given (e.g., "error bars are not reported because it would be too computationally expensive" or "we were unable to find the license for the dataset we used"). In general, answering "[No] " or "[NA] " is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading "NeurIPS Paper Checklist"**,
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers**.

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and Section 1 state four contributions, each tied to a results section; the scope limits (one model family, no head-to-head with ROME, MEMIT or in-context editing) are stated in Section 1 and Section 6.

   Guidelines:

   - The answer NA means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: Section 6 (Limitations) lists the single model family, SAE release and layer, the rank-1 criterion, the single seed, the KL term built into the loss, the stand-in measures and the uncontrolled ROME cross-check.

Guidelines:

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper has no theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Sections 3 and 4 and Appendices A and E give the method, equations, hyperparameters, benchmark construction, splits, arms and metrics.

Guidelines:

- The answer NA means that the paper does not include experiments.
- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: The code and result files are in the project repository, which is not public yet. [TODO: add the public link and a tagged release, then change this answer]

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so "No" is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://nips.cc/public/guides/CodeSubmissionPolicy`) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.

- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 and Appendix A: benchmark construction and splits, hyperparameters, optimiser, and which choices were made on validation prompts.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: Exact paired tests (McNemar, Wilcoxon) and bootstrap intervals are reported where methods are compared. All results come from a single seed, which Section 6 states.

   Guidelines:

   - The answer NA means that the paper does not include experiments.
   - The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
   - The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
   - The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
   - The assumptions made should be given (e.g., Normally distributed errors).
   - It should be clear whether the error bar is the standard deviation or the standard error of the mean.
   - It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
   - For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
   - If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

Justification: Section 4 and Appendix A: GPUs and CPU used, PyTorch and CUDA versions, per-prompt compute times; Appendix A notes a GPU-memory problem that slowed some runs.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The work uses public models and the public CounterFact data and involves no human subjects or personal data.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: Section 6 (Broader impact) discusses auditability and correction without retraining, and the risk that edits steer a model toward arbitrary outputs.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

    Answer: [NA]

    Justification: No new high-risk model or data is released; the models are GPT-2 Small and Medium.

    Guidelines:

    - The answer NA means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

    Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

    Answer: [Yes]

    Justification: GPT-2, TransformerLens, SAELens, the SAE release, Neuronpedia, ROME code and CounterFact are cited; Appendix A states the licences. [TODO: confirm each licence before submission]

    Guidelines:

    - The answer NA means that the paper does not use existing assets.
    - The authors should cite the original paper that produced the code package or dataset.
    - The authors should state which version of the asset is used and, if possible, include a URL.
    - The name of the license (e.g., CC-BY 4.0) should be included for each asset.
    - For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
    - If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
    - For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
    - If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: The code and result packs are documented in the repository (README files, a numbered research journal and a plan with every pass bar). [TODO: confirm once the repository is public]

    Guidelines:

    - The answer NA means that the paper does not release new assets.

- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [NA]

    Justification: No crowdsourcing or human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [NA]

    Justification: No human subjects.

    Guidelines:

    - The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

    Answer: [Yes]

    Justification: The studied models (GPT-2 Small and Medium) are described in Sections 3 and 4. No LLM is used as a component of the method; Appendix A states the use of AI coding assistants during development. [TODO: align with the venue's LLM policy]

    Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (`https://neurips.cc/Conferences/2025/LLM`) for what should or should not be described.